Classification d'images avec des CNN

Dataset Mélanome

Yanis ANDRE, Yanis DJELALI

13 octobre 2026

Plan de l'exposé

1. Le dataset
2. Représentation des données
3. Dataset et DataLoader
4. Premier CNN
5. Normalisation
6. Data augmentation
8. Optimiseur et learning rate
9. Scheduler
10. ResNet pré-entraîné
11. Fine-tuning
12. Analyse des erreurs
13. Conclusion

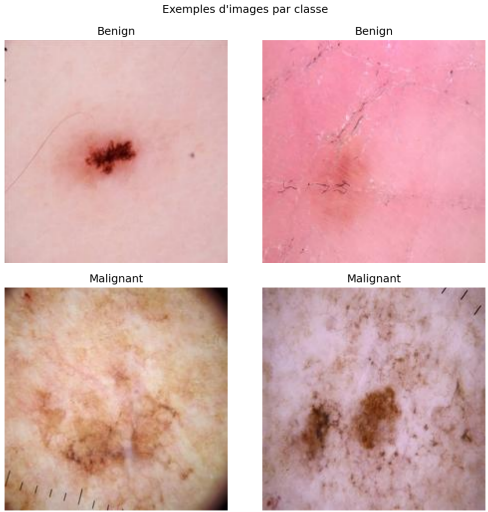
1. Le dataset Mélanome

Présentation

- **Tâche** : classification binaire de lésions cutanées
- **2 classes** : `Benign` et `Malignant`
- **Images** : 224×224 px, RGB
- **Train** : 11 879 images
- **Test (val)** : 2 000 images
- Distribution : 53 % / 47 % (équilibré)

Difficulté

Les deux classes se ressemblent visuellement – même un humain non-expert peut hésiter.



2. Une image pour PyTorch : un tenseur [C, H, W]

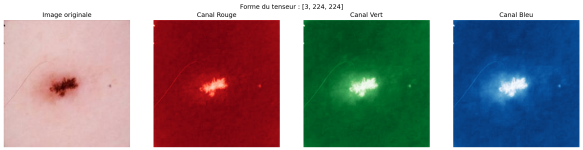
Dimensions

- **C** = 3 canaux (R, V, B)
- **H** = 224 pixels (hauteur)
- **W** = 224 pixels (largeur)

$\Rightarrow$ forme : [3, 224, 224]

ToTensor()

Pixels entiers $[0, 255] \rightarrow$ flottants $[0.0, 1.0]$
(division par 255, convention PyTorch)



3. Dataset et DataLoader

`MelonaDataset` – un annuaire

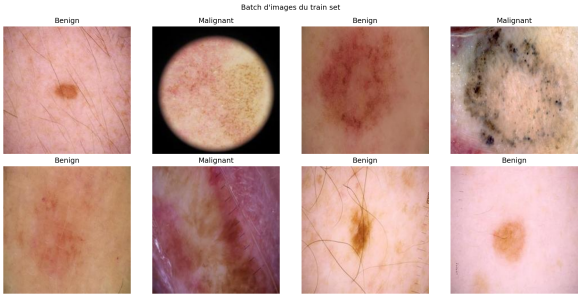
Répond à 2 questions :

- Combien d'images ? → `__len__()`
- Image numéro i ? → `__getitem__(i)`

Charge les images **à la demande**, pas tout en mémoire.

`DataLoader` – le bibliothécaire

- Regroupe en batchs de **32 images**
- `shuffle=True` pour le train
- `shuffle=False` pour la val
- Forme d'un batch : `[32, 3, 224, 224]`



4. Loss, gradient, epoch

CrossEntropyLoss

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}} \qquad \mathcal{L} = -\log(p_y)$$

Plus p_y est faible, plus $\mathcal{L}$ est grande.

Descente de gradient

$$w \leftarrow w - \alpha \cdot \nabla_w \mathcal{L}$$

α = learning rate

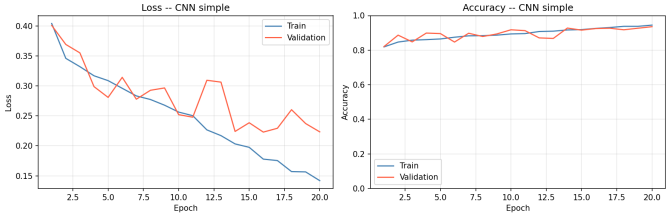
Les 5 étapes par batch

1. `optimizer.zero_grad()`
2. `outputs = model(images)`
3. `loss = criterion(outputs, labels)`
4. `loss.backward()`
5. `optimizer.step()`

Loss vs Accuracy

Loss : valeur continue, sert au gradient.
Accuracy : % bonnes prédictions, lisible humain.

5. Premier CNN – Résultats



Résultats

- **Best val acc : 93,6 %**
- Train acc finale : 94,5 %
- Léger overfitting vers epoch 13

Limites

Convergence lente, val loss instable.
Pixels pas à la bonne échelle.

6. Normalisation – Pourquoi et comment ?

Formule par canal

$$\hat{x} = \frac{x - \mu}{\sigma}$$

μ, σ calculés sur le **train uniquement**

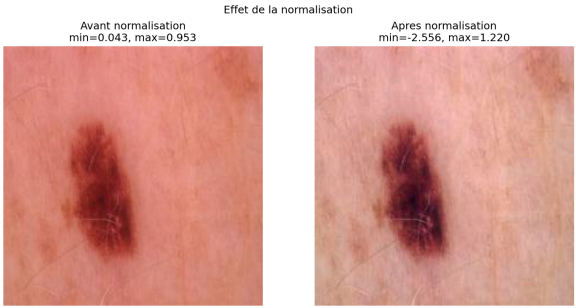
$\Rightarrow$ moyenne ≈ 0, écart-type ≈ 1

Stats obtenues sur Mélanome

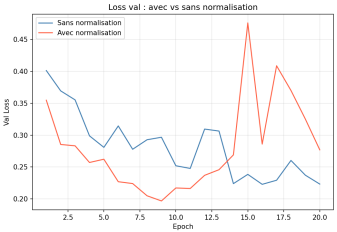
`MEAN` = [0.72, 0.56, 0.54]

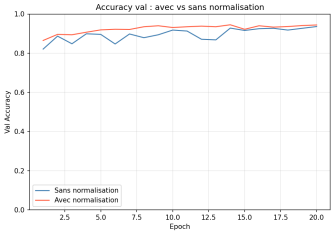
`STD` = [0.19, 0.20, 0.21]

Canal Rouge dominant (cohérent avec la peau).



6. Impact de la normalisation





Impact observé

- Convergence **plus rapide**
- Val acc max : **94,5 %**
- Overfitting plus marqué

Conséquence

Le réseau mémorise trop vite.
$\Rightarrow$ data augmentation nécessaire.

7. Data Augmentation – Lutter contre l'overfitting

Overfitting

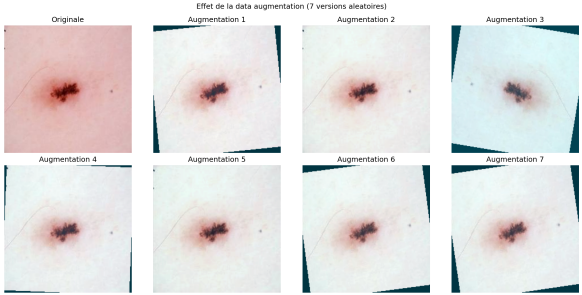
Train loss ↓ mais val loss ↑
⇒ le réseau **mémorise** au lieu de généraliser.

Transformations (train uniquement)

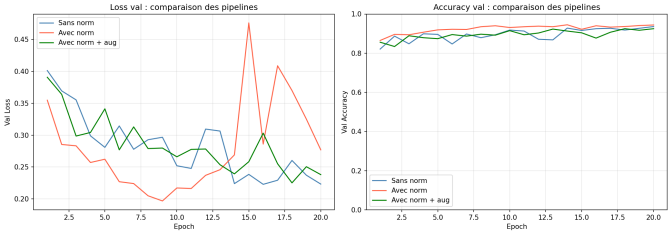
- `RandomHorizontalFlip(p=0.5)`
- `RandomRotation(10°)`
- `ColorJitter(brightness=0.2, contrast=0.2)`

Jamais sur la val !

On mesure sur des images réelles, non altérées.



7. Impact de l'augmentation



Résultats

- Val acc : **92,5 %**
- Train acc : 90 % (au lieu de 99 %)
- Val acc > Train acc !

Le modèle généralise

Léger recul de val acc max, mais aucun overfitting – comportement robuste.

8. Comparaison optimiseurs et learning rates



Configurations testées

- Adam $lr = 10^{-3}$: 90,6 %
- Adam $lr = 10^{-4}$: **92,8 %**
- SGD $lr = 10^{-2}$, mom= 0.9 : 91,8 %

Adam $lr = 10^{-4}$ gagne

Raffinement plus précis.
Adam adapte par paramètre, mais le LR global reste critique.

9. Learning Rate Scheduler – StepLR

Principe

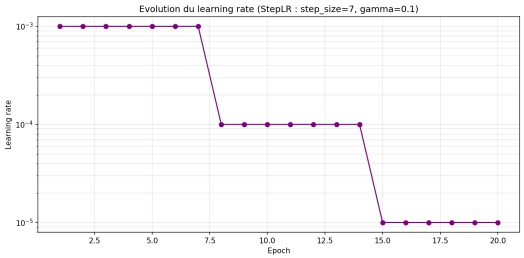
Diviser le LR par 10 toutes les 7 epochs :

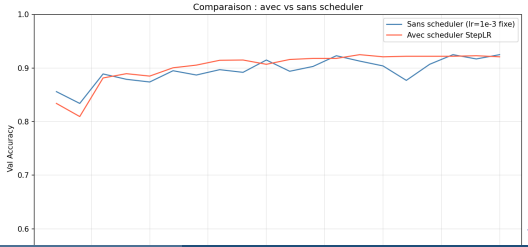
- Epochs 1–7 : $lr = 10^{-3}$ (exploration)
- Epochs 8–14 : $lr = 10^{-4}$ (raffinement)
- Epochs 15–20 : $lr = 10^{-5}$ (finalisation)

Résultat

Best val acc : 92,5 %

Oscillations $< 0{,}2$ pt en fin d'entraînement.



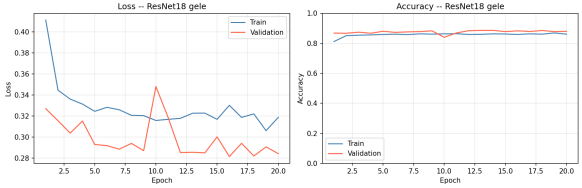


11. Transfer Learning – ResNet18 gelé

Principe

ResNet18 entraîné sur **ImageNet** (1,2M images, 1000 classes).

On gèle tout (`requires_grad=False`) sauf la couche finale.



Paramètres

- Total : 11,2 M
- Entraînables : **1 026** (0,01 %)

Best val acc : 88,6 %

Inférieur au CNN maison !
ImageNet $\neq$ images médicales.

12. Fine-tuning – Dégeler `layer4` et `layer3`

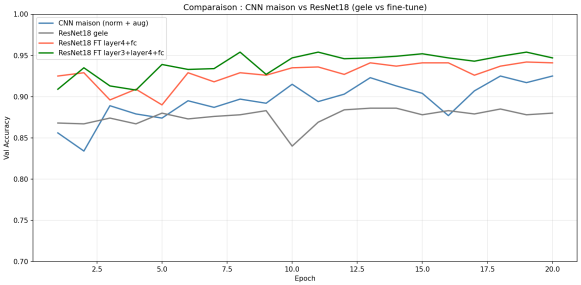
Stratégie avec LR différentiel

- Couches dégelées : $lr = 10^{-4}$
  (déjà entraînées → petits ajustements)
- `fc` (tête) : $lr = 10^{-3}$
  (aléatoire → apprentissage actif)

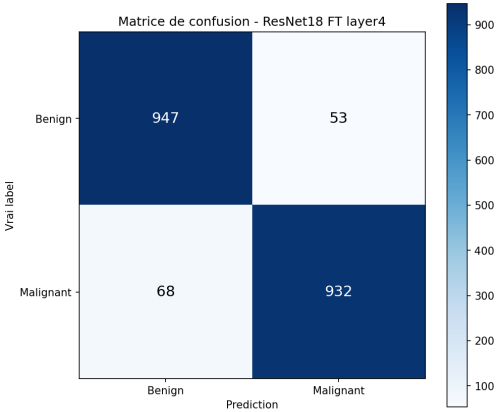
Résultats

- Layer4 + fc : **94,2 %** (8,4M)
- Layer3+4 + fc : **95,4 %** (10,5M)

Layer3 apporte +0,9 pt – gain marginal.



13. Matrice de confusion – ResNet18 fine-tuné



Lecture (sur 2 000 images val)

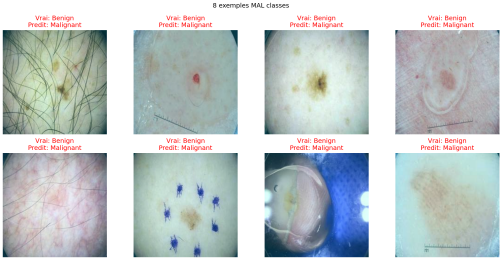
- Vrais Benign → Benign : 947
- Vrais Malignant → Malignant : 932
- Faux positifs : 53
- **Faux négatifs : 68**

Recall Malignant : 93 %

⇒ 7 % de cancers non détectés.
Pour un usage clinique : viser 99 %+.

13. Exemples d'erreurs et de bonnes prédictions

8 erreurs de classification



8 bonnes prédictions



Analyse

Les erreurs portent sur des cas **intrinsèquement ambigus** – difficiles même pour un humain non-expert.

14. Tableau récapitulatif

Modèle / Configuration	Best val acc	Params entraînables
CNN simple	93,6 %	12,9 M
CNN + normalisation	94,5 %	12,9 M
CNN + norm + augmentation	92,5 %	12,9 M
CNN + scheduler	92,5 %	12,9 M
ResNet18 gelé	88,6 %	1 026
ResNet18 FT layer4	94,2 %	8,4 M
ResNet18 FT layer3+4	**95,4 %**	**10,5 M**

14. Conclusion

Modèle retenu : ResNet18 fine-tuné (layer3 + layer4 + fc)

95,4 % de val accuracy – meilleur modèle grâce au transfer learning et à l'adaptation des couches hautes au domaine médical.

Ce qui a le plus amélioré les performances

1. **Transfer learning** : ImageNet apporte une base solide
2. **Fine-tuning partiel** : adaptation au domaine sans tout casser
3. **Normalisation** : convergence plus rapide et stable
4. **Data augmentation** : robustesse et anti-overfitting

Avec plus de temps...

ResNet50 / EfficientNet, augmentation plus agressive, ajustement du seuil de décision pour minimiser les faux négatifs.

Merci de votre attention

Questions ?